

Day 3

Orchestration · HITL · Audit

통합 운영자료 (개념+실습 병행) · 6/10 水 · 09:00–17:00

Supervisor로 5개 Agent를 조율하고, 검증 루프·HITL·감사 로그를 실행한다.

이 파일 하나로 하루 수업을 끝까지 진행합니다. 상세 이혼은 *Appendix*를 참조하세요.

Day 3 운영 흐름

Supervisor로 5개 Agent를 조율하고, 검증 루프·HITL·감사 로그를 실행한다.



Day 3 — 무엇을 배우고 무엇을 만드는가

학습 목표·실습 목표·사용 파일·성공 기준을 시작 전에 꼭 봐야 합니다.

학습 목표 (개념·실습 병행)

- 중앙 오케스트레이션 VS 자율 협업
- State·Node·Edge·조건부 분기
- 검증 루프와 HITL 게이트의 원리

실습 목표 (실습·산출물 확인)

- supervisor_graph 전체 실행
- Router 분기 직접 개조
- --hitl 승인·반려 + audit_log 해부

사용 파일

- orchestrator/supervisor_graph.py
- agents/ (5쪽)
- memory/audit_log.jsonl

성공 기준

- router → ... → report 노드 흐름 출력
- HITL에서 High 이상 발령 시 정지
- audit_log에 5W(누가·언제·무엇·왜·결과) 기록

5일 로드맵: 단일 Agent에서 통합 Security PoC까지

오늘이 전체 과정에서 어디에 위치하는지 먼저 확인합니다.

단일 Agent에서 통합 Security PoC까지 이어지는 하나의 파이프라인입니다.



① 개념 설명 → ② 파일 확인 → ③ 직접 실행 → ④ 산출물 확인 → ⑤ 관찰 정리 → ⑥ 다음 연결

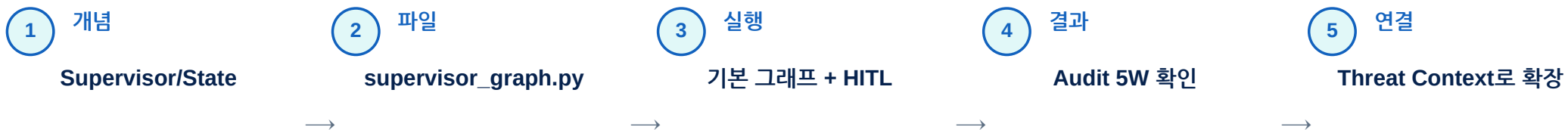
개념 + 실습 병행 운영 패턴 (09:00-17:00)

개념을 설명하고 바로 파일 확인·실행·산출물 검증으로 이어갑니다.

수업은 이론을 몰아서 듣는 방식이 아니라, 작은 학습 루프를 하루에 3회 이상 반복합니다.



매일 최소 3회 이상 직접 실행



운영 메모: DDD 관점: Agent를 도메인 책임 단위로 나누고, State를 통해 책임 경계를 유지한다.

오늘의 자료 구조: PPT → 코드 → 명령 → 산출물

수강생이 지금 보는 장표가 어떤 파일과 어떤 결과물로 이어지는지 명확히 보여줍니다.

PPT의 개념은 실제 Codespaces 파일·명령어·산출물로 바로 연결되어야 합니다.



PPT 개념

여러 Agent를
조율하는
Supervisor

Supervisor · State · HITL

확장 개념
Sub-Agent, Review Loop,
State Graph



관련 파일

orchestrator/supervisor_graph.py

agents/*.py

memory/audit_log.jsonl



실행 명령

```
python orchestrator/supervisor_graph.py --target  
sample_app/
```

```
python orchestrator/supervisor_graph.py --target  
sample_app/ --hitl
```



산출물

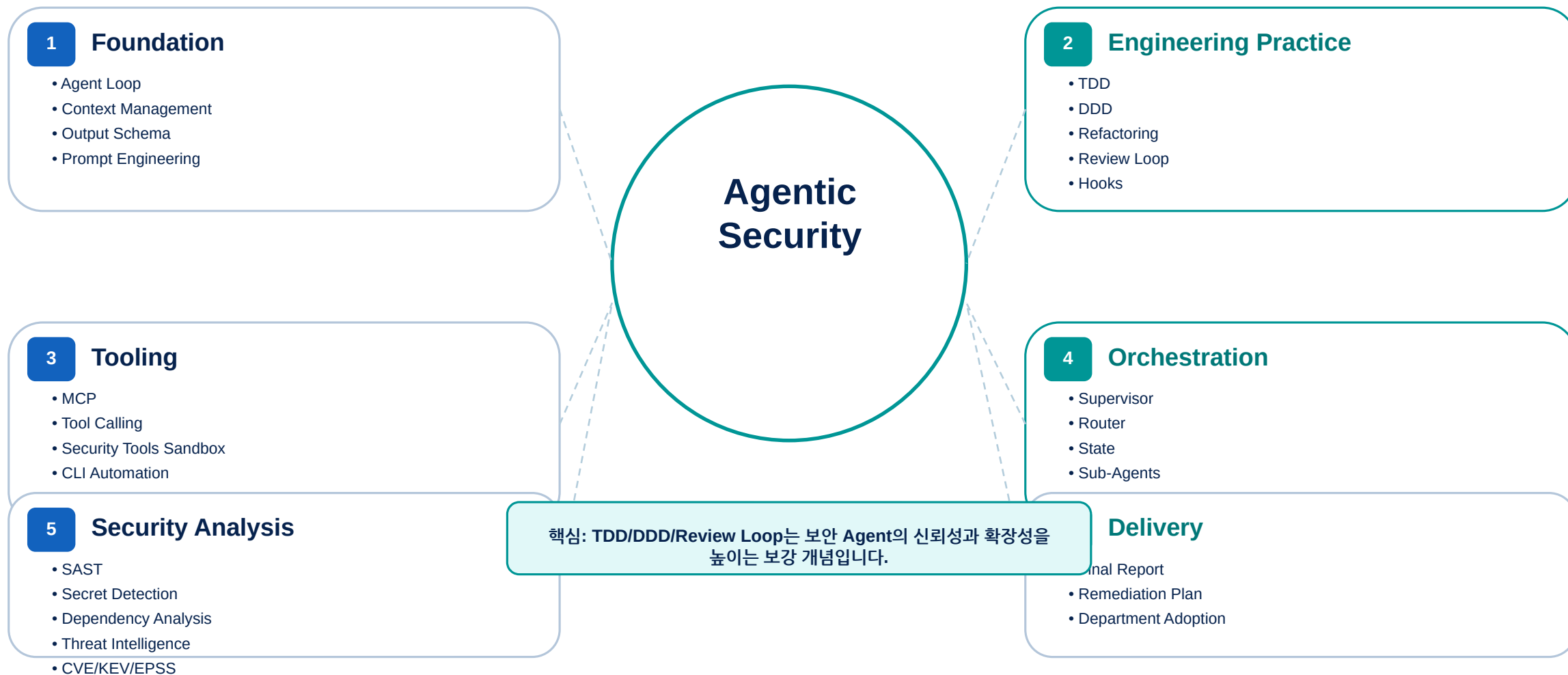
reports/lab3_report.md

memory/audit_log.jsonl

보강 개념 맵: 실전 에이전틱 보안 엔지니어링

TDD·DDD·Review Loop·Hooks 등은 현재 실습 흐름을 보강하는 개념으로 연결합니다.

레퍼런스의 실전 에이전틱 코딩 패턴을 현재 보안 실습 흐름에 맞게 재구성합니다.



여러 Agent를 조율하는 Supervisor

이 그림은 오늘의 핵심 구조를 파일·실행·산출물 관점으로 연결합니다.



Mini Lab → Main Lab 연결

작은 실험으로 개념을 먼저 체감하고, 바로 보안 실습으로 확장합니다.

Mini Lab · Bug Hunt Game

Planner
Hunter
Reviewer
Reporter

Main Lab · Security Supervisor

Supervisor
State
HITL
Audit Log

연결 문장

Planner/Hunter/Reviewer/Reporter 역할 분리는 SAST/Secret/Dependency/Threat/Report Agent 구조로 확장됩니다.

Mini Lab 실행 · Bug Hunt Game

한 줄 명령으로 실행하고, 산출물을 확인합니다.

왜 하는가

멀티에이전트는 모델 수가 아니라 역할과 책임을 나누는 구조임을 체감합니다.

```
python course/mini_labs/day03_multi_agent_game/run_bug_hunt_game.py
```

관련 파일

```
course/mini_labs/  
day03_multi_agent_game/  
run_bug_hunt_game.py  
bug_snippet.py  
roles.md
```

생성 산출물

```
reports/bug_hunt_report.md
```

확인 포인트

역할별 결과 분리
Reviewer 검증 이유
Supervisor 본실습 연결

Sub-Agent · Review Loop · State

부가 개념을 키워드가 아니라 실습·산출물·검증과 연결합니다.

Sub-Agent

큰 일을 작은 역할로 나눕니다.
각 Agent는 좁고 명확한 책임을 가집니다.

Review Loop

결과를 바로 믿지 않고 검증합니다.
부족하면 재시도하고 위험하면 HITL로 멈춥니다.

State

Agent 간 직접 대화가 아니라 중앙 상태를 경유합니다.
감사와 재현성을 확보합니다.

단일에서 멀티로 왜 지휘자가 필요한가

Day 1에서 본 단일 Agent의 한계를 다시 떠올립니다. 분업의 답이 곧 오케스트레이션입니다.

①

역할 분리

SAST·시크릿·의존성·위협·리포트를 각 전문 Agent가 맡으면, 각자의 근거 품질이 올라갑니다.

②

순서 통제

의존성이 CVE를 찾아야 위협 Agent가 조회할 수 있습니다. 누가 먼저인지를 지휘자가 정합니다.

③

검증·감사 지점

모든 결과가 증량을 지나야, 검증·승인·기록을 한곳에 둘 수 있습니다. 분산 구조에는 이 자리가 없습니다.

본 과정은 '허브앤스포크(중앙 Supervisor)'를 채택합니다. 중앙 집중이 통제·감사를 가능하게 하기 때문입니다.

큰 일은 쪼개서 위임한다 서브에이전트 사고법

에이전트 코딩의 핵심 사고법입니다. 메인 흐름의 책상(컨텍스트)을 아끼려던 하위 작업을 떼어 맡깁니다.

한 에이전트가 다 할 때

- 긴 탐색·분석 이력이 책상을 차지
- 본 작업의 맥락이 뒤로 밀림
- 권리와 집중이 함께 떨어짐

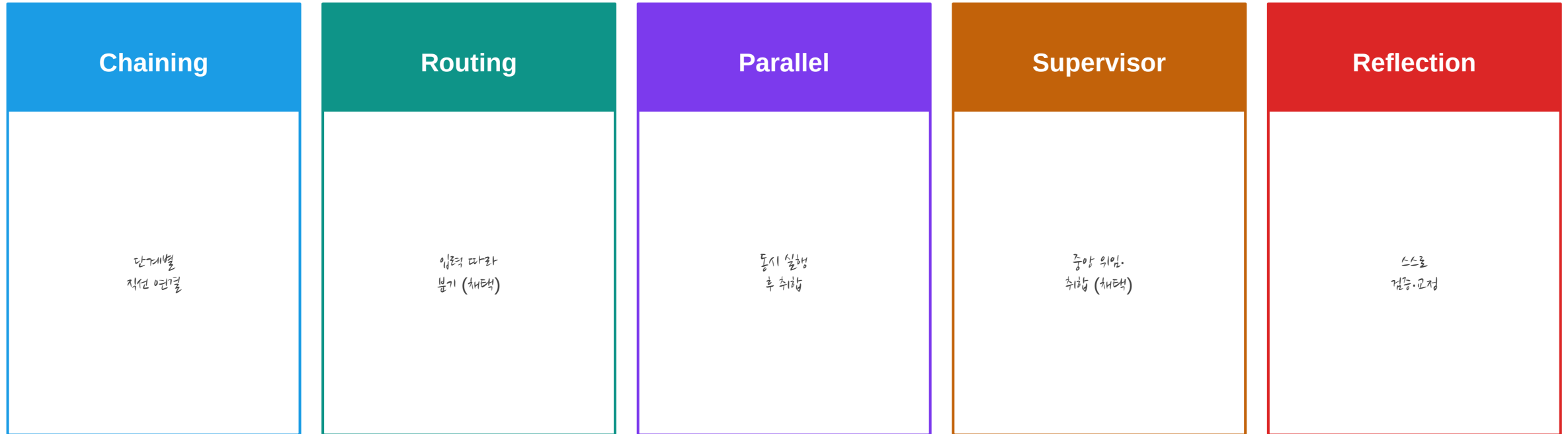
하위 작업을 위임할 때

- 하위 에이전트가 자기 책상에서 수행
- 메인에는 '요약 결과'만 돌아옴
- 각자 좁고 깊게 — 분업의 본질

우리 Supervisor 구조가 정확히 이 사고법입니다. 다섯 전문 Agent가 각자의 일을 하고, 중앙에는 정제된 결과만 모입니다.

워크플로우 5대 패턴 일을 잇는 방식

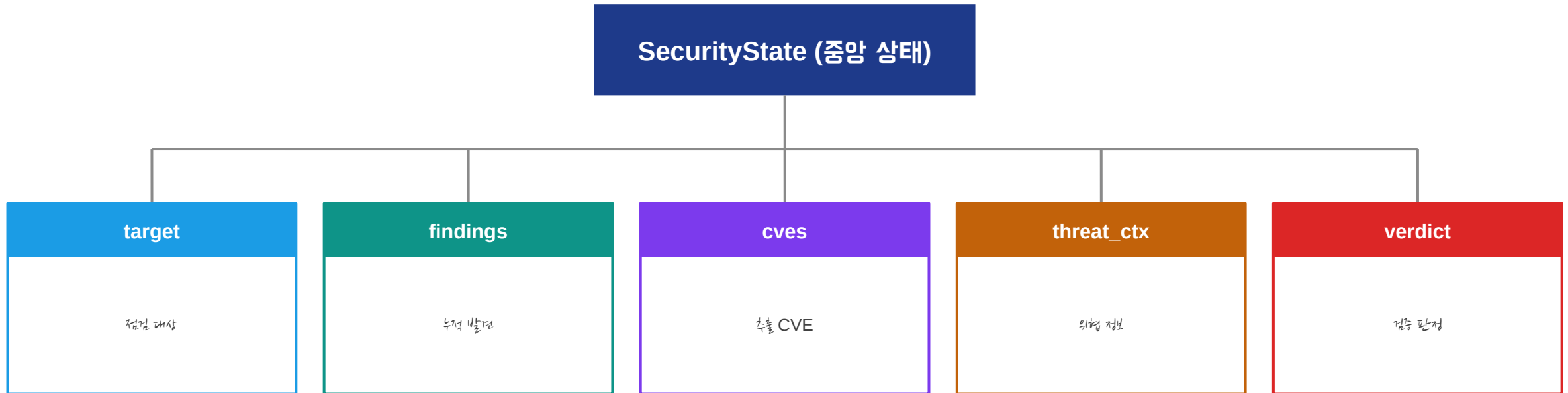
여러 Agent를 잇는 방식에는 대표적인 다섯 패턴이 있습니다. 우리는 이 중 둘을 조합합니다.



우리 supervisor_graph는 Routing(입력별 경로 분기)과 Supervisor(중앙 조율)를 결합하고, 검증 노드에 Reflection을 살짝 더합니다.

LangGraph의 핵심 모든 것은 State를 지난다

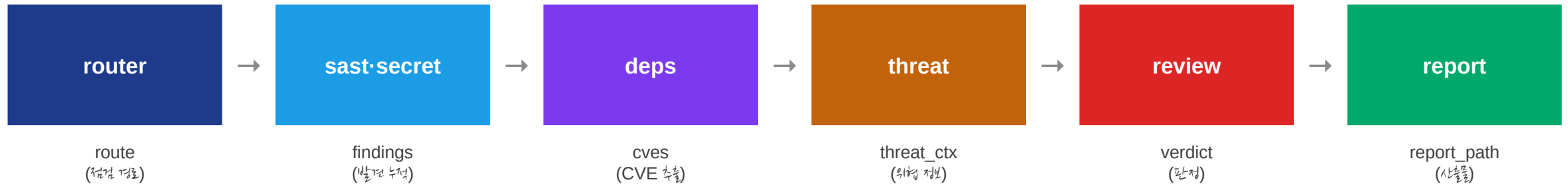
오케스트레이션의 핵심은 '중앙 상태(State)'입니다. Agent들은 직접 대화하지 않고 State를 통해서만 주고받습니다.



Agent 간 직접 호출은 금지입니다. 모든 전달은 State를 경유하므로, State만 보면 '지금까지 무슨 일이 있었는지' 전부 알 수 있습니다.

그림으로 보는 State — 누가 무엇을 쓰는가

각 노드가 중앙 State에 무엇을 적어 넣는지 순서대로 따라가면, 그래프 전체가 한눈에 들어옵니다.



디버깅 요령: 그래프가 이상하면 이 그림 순서대로 State 필드를 확인하세요. 어느 칸이 비어 있는지가 곧 고장 지점입니다.

검증 루프와 HITL — 멈춤과 기록의 원리

자동화의 신뢰는 '잘 도는가'가 아니라 '위험할 때 멈추고 기록하는가'에서 옵니다. 하네스 요소 ⑤입니다.

검증

PASS / RETRY / BLOCK

결과가 기준을 만족하면 PASS, 부족하면 RETRY(상한 2회), 심각·정책 위반이면 BLOCK. 이 판정이 그래프 경로를 바꿉니다.

HITL

사람 승인 강제

High 이상 발견 전 앞에서 파이프라인이 멈추고 사람의 승인을 받습니다. 기본값은 거부(deny-by-default)입니다.

감사

모든 결정 기록

라우팅·판정·승인을 시계열로 audit_log.jsonl에 남깁니다(append-only). 누가·언제·무엇을·왜·결과의 5W가 남습니다.

그림으로 보는 진화 단일에서 Supervisor까지

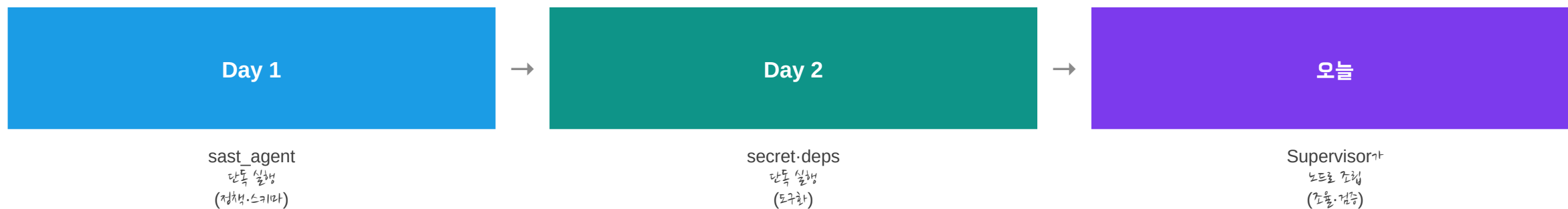
왜 중앙 지휘자가 필요한지 한 장으로 봅니다. 에이전트를 늘리는 것만으로는 부족하고, 조율 구조가 있어야 합니다.



핵심은 '몇 개'가 아니라 '어떻게 조율하는가'입니다. 중앙 Supervisor가 있어야 검증·HITL·감사를 한곳에 둘 수 있습니다.

어제까지의 부품이 오늘의 노드가 된다

Day 1~2에서 단독 실행한 Agent들이 Day 3 그래프의 노드로 그대로 들어갑니다. 새 개념이 아니라 조립입니다.



단독 실행 때와 같은 함수가 그대로 노드가 됩니다. 달라지는 것은 '누가 언제 부르고, 결과를 어디에 모으는가'뿐입니다.

Sub Agent와 Review Loop — 위임과 자기검증

에이전틱 코딩에서 널리 쓰이는 두 패턴입니다. 우리 Supervisor 구조가 이 패턴의 보안 버전입니다.

Sub Agent

하위 작업 위임

메인 흐름이 모든 일을 떠안지 않고, 독립된 하위 에이전트에 작업을 떼어 맡깁니다. 메인에는 정제된 결과만 돌아와 전텍스트가 깨끗하게 유지됩니다.

Review Loop

자기 검증 순환

결과를 그대로 받아들이지 않고, 별도의 검증 단계가 기준 충족 여부를 판정합니다. 부족하면 다시 분석시킵니다 — 우리 PASS/RETRY/BLOCK이 이 패턴입니다.

연결

보안 버전

우리 5개 Agent는 Sub Agent 패턴의 보안 특화이고, review 노드는 Review Loop의 구현입니다. 같은 발상을 차량보안 점점에 옮긴 것입니다.

이 두 패턴은 특정 책이나 도구의 발명이 아니라, 에이전틱 시스템 설계의 공통 어휘입니다. 본 과정은 이를 보안 점검에 적용합니다.

감사 노트 단일과 멀티, 무엇이 진짜 차이인가

수강생들이 자주 혼동하는 지점입니다. '모델을 여러 개 쓰는 것'과 '역할을 나누는 것'은 다릅니다.

흔한 오해

- '멀티 = 모델을 여러 개 띄우는 것'
- '에이전트가 많을수록 똑똑하다'
- '형성하면 자동으로 품질이 오른다'

실제

- 멀티 = 역할과 책임을 나누는 것
- 조율 없는 다수는 오히려 혼란
- 중앙 통제가 있어야 품질이 오름

감사 팁: 같은 LLM 하나로도 역할만 나누면 멀티 에이전트입니다. 핵심은 '몇 개'가 아니라 '어떻게 나누고 조율하는가'입니다.

여기까지 — 개념·실습 병행 미니 점검

여기까지 이해했는지 스스로 점검해 봅니다.



단일에서 멀티로 가는 세 가지 이유를 든다



5대 패턴 중 본 과정이 채택한 둘을 안다



State를 경유해야 하는 이유를 설명한다



검증 3판정(PASS/RETRY/BLOCK)을 구분한다

네 가지에 답할 수 있으면 실습·산출물 확인(LAB 3)으로 진행합니다.

실습 루프 (LAB 3)

LAB 3 — Supervisor로 5개 Agent 조율

이론을 코드로 옮깁니다. 어제까지 만든 도구-Agent를 중앙 그래프가 지휘하게 합니다.

LAB 3가 하는 일 한눈에

실습 첫 실습의 목표, 준비물, 사용 파일, 성공 신호입니다.

목표

- 5개 Agent를 중앙에서 조율
- Router 분기 직접 개조

준비물

- Day 1~2 산출물(정책·도구)
- langgraph 설치(웹이면 fallback)

사용 파일

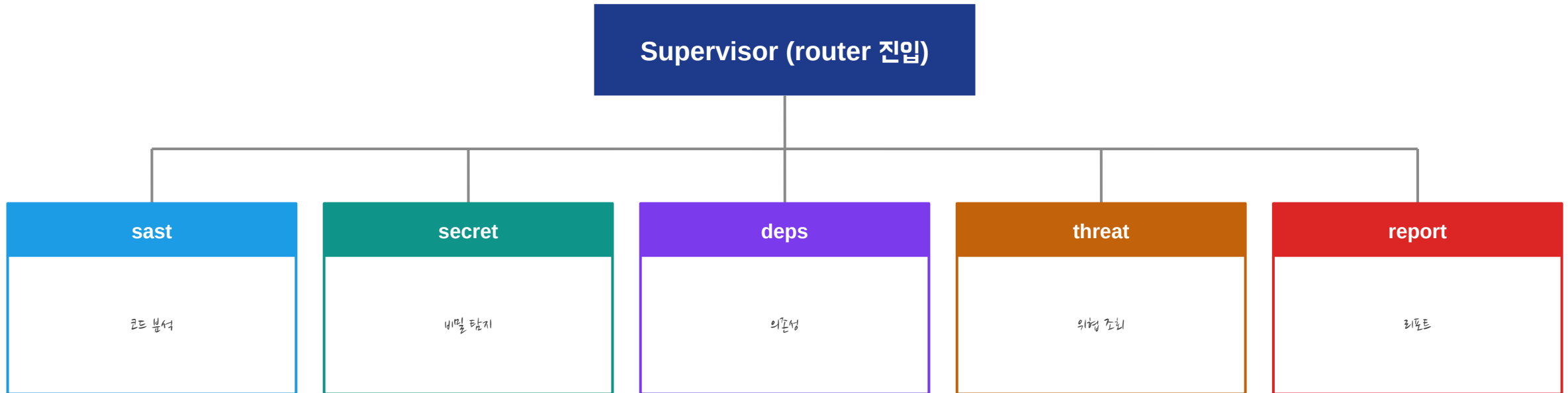
- orchestrator/supervisor_graph.py
- agents/ 5개

성공 신호

- router → report 흐름 출력
- lab3_report.md 생성

supervisor_graph의 구조 노드와 배선

수강생이 손대는 곳(노드 내부)과 손대지 않는 곳(배선)을 분명히 구분합니다.



손대는 곳: route_input 분기·각 노드 프롬프트. 손대지 않는 곳: add_node·add_edge·compile (배선은 템플릿 고정).

전체 파이프라인 실행 ZIP 명령 1:1

정책 파일은 이거에 만든 CLAUDE.md를 그대로 씁니다. 명령은 ZIP 실제 파일과 일치합니다.

실행

```
python orchestrator/supervisor_graph.py --target sample_app/
```

콘솔 출력 (노드 전환 흐름)

```
[router] route=[sast, secret, deps, threat, report]
[sast]   findings 적재
[deps]   CVE 4건 추출 → State
[reviewer] verdict=PASS
[report] reports/lab3_report.md 생성
```

콘솔의 노드 전환 순서가 곧 그래프가 살아 움직이는 모습입니다. Day 1 단일 SAST와 비교해 협업의 효과를 봅니다.

Router 개조 과제 분기 한 줄의 힘

route_input 함수의 분기 조건을 바꿔, 입력에 따라 경로가 달라지는 것을 직접 만듭니다.

과제1 control/ 이중 점검
sample_app/control/ 경로는 sast와 secret을 모두 거치도록 분기를 추가합니다. 차량 제어 모듈을 더 엄격히 보는 것입니다.

과제2 CVE 직행
입력에 CVE 식별자가 있으면 threat 노트로 직행하는 분기를 추가합니다. 이미 아는 취약점은 위협 조회부터 합니다.

확인 경로 변화 관찰
개조 후 [router] 로그의 route 값이 실제로 달라지는지 확인합니다. 배선이 아니라 분기 조건만 바꿨음에 주목합니다.

워크드 비교 같은 대상, 단일 vs 멀티

Day 1 단일 SAST와 오늘 멀티 협업의 결과를 같은 대상으로 비교합니다. 협업의 효과가 숫자로 드러납니다.

Day 1 단일 SAST (vulnerability.py)

발견 8건 – SAST 한 종류만

Day 3 멀티 협업 (sample_app 전체)

발견 16건 – SAST + Secret + Dependency 협업
SAST 9건 + Secret 3건 + Dependency 4건
→ 각 전문 Agent가 자기 영역을 깊이 봄

단일이 놓친 비밀·의존성 취약점을 전문 Agent들이 잡아냅니다. 이것이 '역할 분리'의 구체적 효과입니다.

State를 따라가며 디버깅하기

그래프가 멈추거나 이상하면, State를 출력해 '어디까지 무엇이 담겼는지' 추적합니다.

①

노드별 State 확인

각 노드 진입·종료 시 findings.cves의 개수를 로그로 찍어, 어느 노드에서 비는지 봅니다.

②

흔한 단절 지점

deps가 cves를 State에 안 넣으면, threat가 빈 입력을 받습니다. State 전달이 끊긴 곳을 찾습니다.

③

배선 복원

배선을 잘못 건드리 KeyError가 나면, git checkout으로 supervisor_graph를 템플릿 상태로 되돌립니다.

실습 루프 (LAB 4)

LAB 4 — HITL · 검증 · Audit Log

자동화에 '검증'과 '기록'을 심습니다. 위험 항목 앞에서 사람의 승인을 강제합니다.

HITL 모드 실행 위험 앞에서 멈춘다

--hitl 플래그를 켜면 High 이상 발견에서 파이프라인이 멈추고 승인을 기다립니다.

실행

```
python orchestrator/supervisor_graph.py --target sample_app/ --hitl
```

승인 대기 (대화형 실행 시)

```
[HITL] 승인 대기 - High SQL Injection 의심 (vulnerability.py:39)
'확정'으로 포함할까요? [approve / reject <사유>]:
→ 비대화형(자동 채점) 환경에서는 기본값 거부
```

원칙: 기본값은 거부(deny-by-default), 승인 없이 무회 불가, 모든 결정은 기록. 안전이 편의보다 우선합니다.

검증 3판정 PASS · RETRY · BLOCK

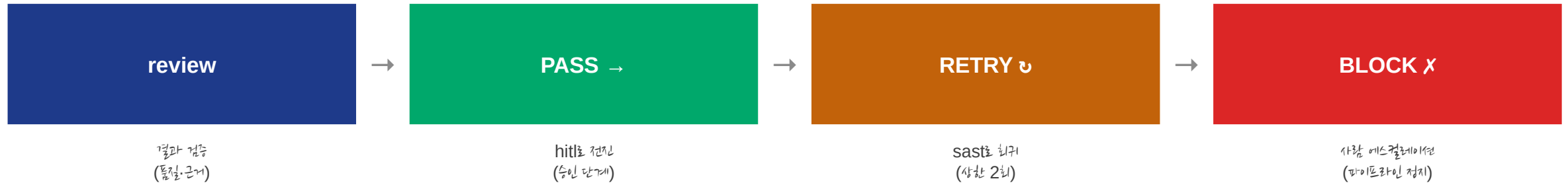
검증 노드가 내리는 세 가지 판정이 그래프의 다음 경로를 결정합니다.

✓ PASS	↺ RETRY	✗ BLOCK
기준 충족 → 다음 단계로	일부 미달 → 재분석(상한 2회)	심각·정책 위반 → 사람에게 에스컬레이션

RETRY에 상한(2회)이 없으면 무한 루프에 빠집니다. 안전장치에도 안전장치가 필요합니다.

그림으로 보는 검증 분기 판정이 경로를 바꾼다

review 노드의 세 판정이 그래프를 어디로 보내는지 시각화합니다. 조건부 Edge의 실체입니다.



같은 그래프라도 판정에 따라 매번 다른 길을 갑니다. '경로가 데이터에 반응한다'는 것이 단순 스크립트와 오케스트레이션의 차이입니다.

Audit Log 해부 5W가 남는가

모든 결정이 시계열로 기록됩니다. 한 줄에서 누가.언제.무엇을.왜.결과를 얻을 수 있어야 합니다.

감사 로그 확인

```
tail -3 memory/audit_log.jsonl
```

레코드 예시

```
{"ts":"2026-06-10T14:22:05","agent":"hitl","action":"decision",  
  "item":"SQL Injection 의심","decision":"reject",  
  "by":"admin","reason":"non-interactive default-deny"}
```

감사 로그는 append-only입니다. 고치거나 지우지 않습니다. 이 불변성이 규제 대응(R155 맥락)의 신뢰 근거입니다.

그림으로 보는 HITL·Audit 타임라인 한 발견의 일생

High 발견 하나가 승인 게이트를 지나 기록으로 남기까지의 시간 순서를 따라가봅니다.



이 다섯 칸이 '통제된 자동화'의 심장입니다. 멈춤(②③)과 기록(④)이 있어야 자동화를 신뢰할 수 있습니다.

확정과 반려를 리포트에서 구분한다

HITL 결정 결과가 리포트에 어떻게 반영되는지 봅니다. 반려된 발견이 대응 대상에 속하면 안 됩니다.

①

HITL 상태 컬럼

risk_score_table.md에 'HITL 상태' 컬럼이 있어, 각 발견이 확정인지 반려/보류인지 한눈에 보입니다.

②

확정/반려 분리

final_report.md는 '확정 발견(대응 대상)'과 '반려/보류 발견(HITL 미승인)'을 별도 섹션으로 나눕니다.

③

비대화형 기본 거부

자동 채점 환경에서는 High 이상이 모두 반려로 처리됩니다. 이는 의도된 안전 동작입니다.

수강생이 '위험도 1순위인데 왜 반려?'라고 물으면, 위험도는 '우선순위 권고'이고 HITL 상태는 '대응 승인 여부'로 다른 축임을 설명합니다.

LAB 3·4 회고 우리 업무로 가져가기

오케스트레이션과 안전장치들을 부서 업무와 연결하는 회고입니다.

관찰한 것

- 모든 결과가 State를 지나야 통제 가능
- 검증·HITL·감사는 한곳에 모여야 함
- 기본값 거부가 안전의 출발점

가져갈 질문

- 우리 점검 흐름에서 사람 승인이 꼭 필요한 지점은?
- 어떤 결정을 반드시 로그로 남겨야 하나?
- 무한 루프·폭주를 막을 상한은 어디에?

여기서 적은 답이 Day 5 통합 PoC와 부서 적용 구상으로 이어집니다.

감사 노트 HITL은 자동화의 적인가

수강생이 'HITL이 자동화를 느리게 한다'고 느낄 때 답할 내용입니다.

오해

- '사람이 끼면 자동화가 아니다'
- '멈추면 효율이 떨어진다'

관점 전환

- 위험한 결정에만 멈춤 → 효율 손실 최소화
- 신뢰할 수 없는 빠름은 무의미
- 사람의 승인이 곧 책임의 소재

감사 팁: 차량 안전 영역에서는 '빠른 자동 대응'보다 '검증된 신중한 대응'이 우선입니다. HITL은 그 신중함의 장치입니다.

여기까지 — 실습·산출물 확인 미니 점검

여기까지 이해했는지 스스로 점검해 봅니다.

☒ 단일 8건 vs 멀티 16건의 차이를 설명한다

☒ State가 끊기는 지점을 추적할 수 있다

☒ PASS/RETRY/BLOCK이 경로를 바꾸는 것을 안다

☒ deny-by-default가 왜 안전한지 설명한다

☒ audit_log에서 5W를 읽을 수 있다

다섯 가지에 답할 수 있으면 오늘의 목표를 달성한 것입니다.

Day 3을 한 장으로 통제의 세 기둥

오늘 다룬 내용을 세 기둥으로 묶습니다. 모두 '통제'를 위한 장치입니다.

1

오케스트레이션

중앙 Supervisor가
역할 순서를 통제

2

검증 루프

PASS/RETRY/BLOCK으로
동작을 통제

3

HITL·감사

법검과 기록으로
위협과 책임을 통제

세 기둥 모두 Day 1에서 배운 '하네스 요소⑤ 안전장치'의 구체적 구현입니다. 오늘 그것이 코드가 되었습니다.

Day 3 — 오늘 만든 산출물 확인

오늘 저장소에 실제로 남은 것들입니다. 다음 Day의 입력적으로 누적됩니다.

01

reports/lab3_report.md

5개 Agent 협업의 통합 점검 리포트 (HITL 상태 점검 포함)

02

memory/audit_log.jsonl

라우팅·판정·HITL 결정의 시계열 기록 (append-only)

멀티 에이전트 + 검증 + 감사 체계가 완성됩니다. Day 4에서 위협 정보를 결합하면 우선순위가 생깁니다.

Day 3 — 자주 나는 오류

강의장에서 가장 흔히 막히는 지점과 즉시 대응법입니다.

KeyError: 'findings'

→ 그래프 버전 훼손 → git checkout으로 템플릿 복원

--hit인데 안 멈춤

→ High 발견된 존재 여부·review 임계값 확인

RETRY 무한 반복

→ 재시도 상한(2회) 코드 누락 확인

audit_log 비어 있음

→ memory/ 디렉터리 쓰기 권한 확인

Day 3 — 완료 체크리스트 · 다음 Day 연결

체크리스트로 마감하고, 내일로 이어지는 연결고리를 확인합니다.

☒ supervisor_graph 전체 실행 성공

☒ router → report 흐름 확인

☒ Router 분기 개조 적용

☒ --hitl에서 High 발견 정지 확인

☒ audit_log 5W 기록 확인

☒ 확장/반려 발견이 리포트에서 구분되는지 확인

다음 Day 연결고리 → Day 4: 발견은 많은데 '무엇부터?'에 답하려면 외부 위협 정보(CVE·KEV·EPSS)가 필요하다

APPENDIX

부록 이론 상세 보충

본문에서 압축한 개념의 상세 설명입니다. 수업 중 질문이 나오거나, 복습 시 참조하세요.

[상세] 워크플로우 5대 패턴 비교

본문 '이론 2'의 상세판. 다섯 패턴을 같은 기준으로 표정합니다.

패턴	구조	장점	본 과정 사용
Chaining	단계적 직선	예측 가능	부분 (순차 흐름)
Routing	입력별 분기	흐름적 배분	채택 (route_input)
Parallel	동시 실행	빠름	미사용 (단순화)
Supervisor	중앙 위임	통제·감사	채택 (하위)
Reflection	자기 검증	품질 향상	부분 (review 노트)

단순함이 곧 통제 가능성입니다. 본 과정은 Parallel을 일부러 빼고 순차+중앙 구조를 택해, 검증·감사 지점을 분명히 했습니다.

[상세] State·Node·Edge — LangGraph 3요소

본문 '이론 3'의 상세판. 그래프를 이루는 세 부품을 정의합니다.

State **중앙 상태**
모든 노드가 공유하는 데이터 묶음(SecurityState). 노드는 State를 읽고, 수정한 State를 돌려줍니다.

Node **작업 단위**
하나의 일을 하는 함수. 우리의 sast·secret·deps·threat·review·report가 각 노드입니다.

Edge **연결**
노드 간 경로. 고정 Edge(항상 같은 다음)와 조건부 Edge(판정에 따라 갈림)가 있습니다. 배선은 템플릿 고정.

[상세] Provenance — 발견의 출처 추적

본문 LAB 4의 배경. '이 발견이 어디서 왔는가'를 끝까지 되짚는 능력입니다.

①

도구까지 역추적

각 발견의 provenance 필드(tool:sast_rules#...)로 어느 도구·규칙이 만들었는지 추적합니다.

②

결정까지 역추적

audit_log로 그 발견이 승인됐는지 반려됐는지, 누가 언제 결정했는지 되짚습니다.

③

왜 중요한가

보안 사고 사후 조사에서 '왜 이 판단을 했는가'를 증명할 수 있어야 합니다. Provenance가 그 증거 사슬입니다.

Provenance(출처 증명)는 자동화 신뢰의 핵심입니다. 결과만 있고 출처가 없으면, 그 결과를 믿을 근거가 없습니다.

[상세] deny-by-default — 왜 기본값이 거부인가

본문 LAB 4 HITL의 원리. 안전 설계의 기본 철학입니다.

원리

모르면 거부

승인 응답이 없거나(타임아웃), 불명확하면 '궁뎅'이 아니라 '거부'가 기본입니다. 침묵을 동의로 해석하지 않습니다.

이유

실패의 방향

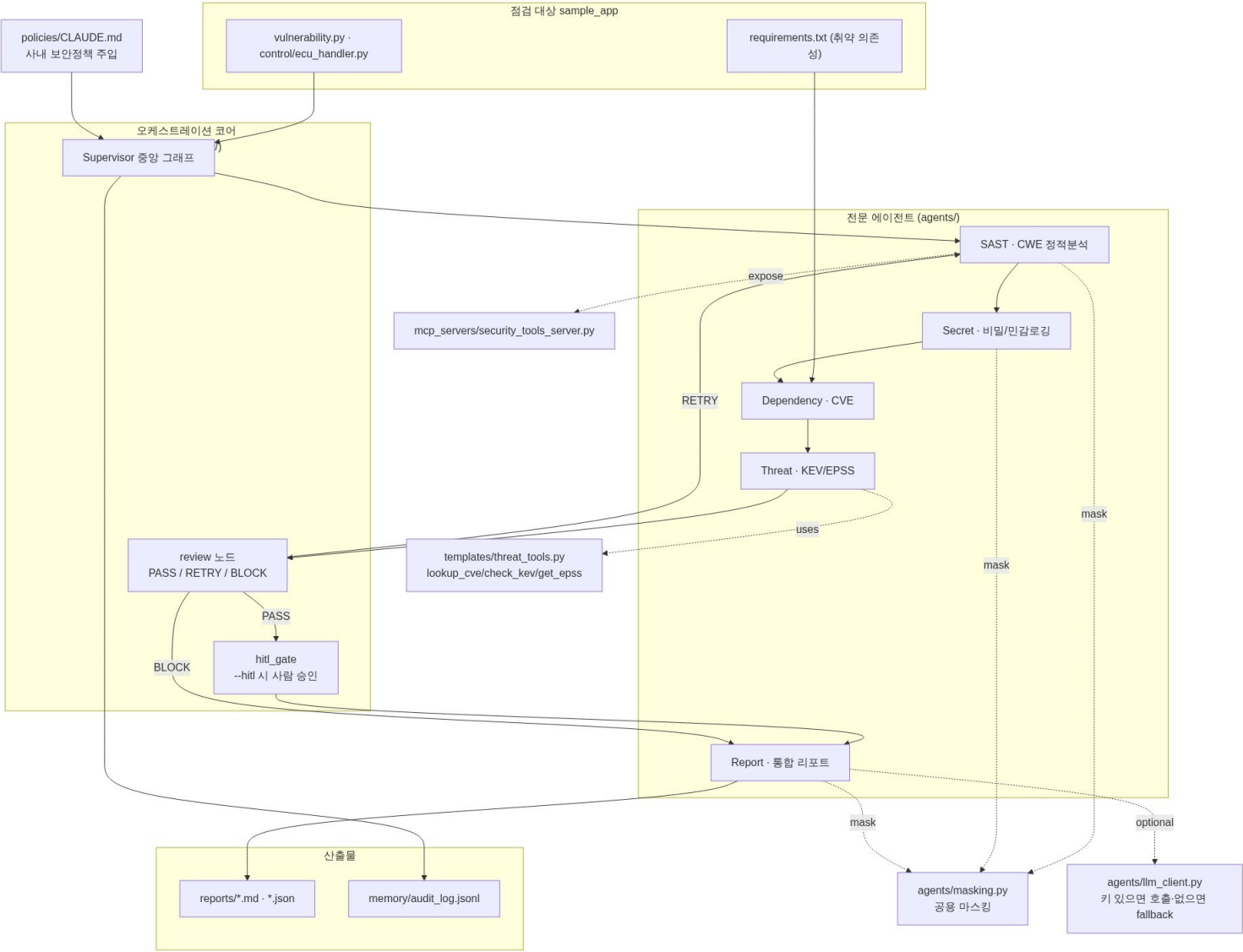
시스템이 실패하더라도 '안전한 쪽'으로 실패해야 합니다(fail-safe). 거부로 실패하면 위험이 통과하지 않습니다.

적용

비대화형 채점

자동 채점 환경에서 High 이상이 모두 반려되는 것은, 사람 승인이 없었기 때문입니다. 의도된 안전 동작입니다.

전체 아키텍처 — Security Agent Lab



Day 3 동작 흐름 (시퀀스 다이어그램)

